Lesson 08 Demo 01

Testing a Document Object Model Using Jest

Objective: To test and manipulate a document object model (DOM) using Jest and jsdom in a Node.js environment to verify DOM manipulations

Tools required: Visual Studio Code, Node.js, and npm

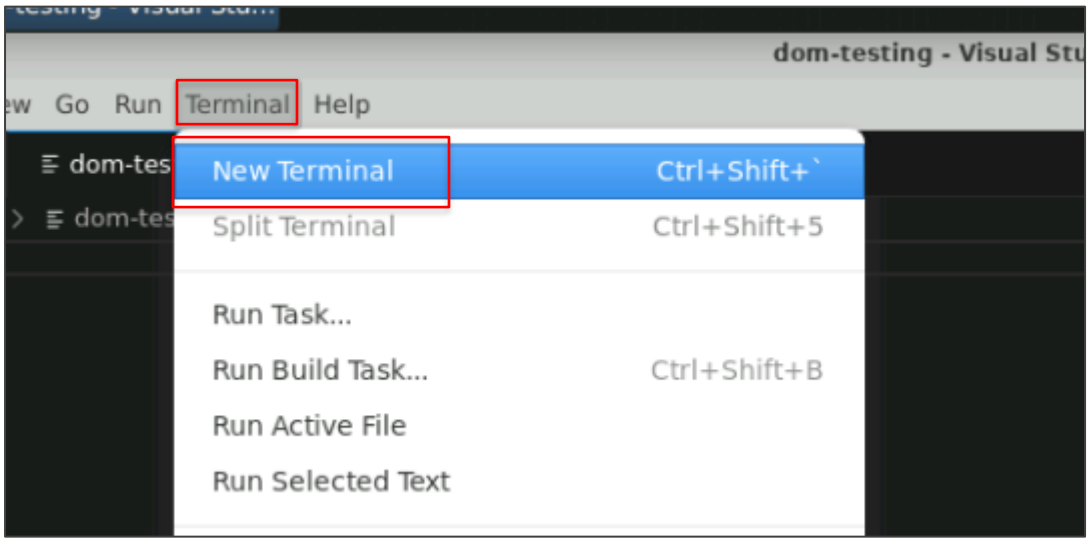
Prerequisites: A good understanding of JavaScript, HTML, and DOM, as well as some knowledge of Jest

Steps to be followed:

1. Create a Node project
2. Create an HTML file
3. Create a JavaScript file
4. Write a Jest test
5. Run the Jest test

Step 1: Create a Node project

1.1 Open Visual Studio Code, click on **Terminal**, and select **New Terminal**



1.2 Use the following command to create a folder named **dom-testing** on the Desktop:

cd Desktop

mkdir dom-testing

```
labuser@ip-172-31-44-9:~$ cd Desktop/
labuser@ip-172-31-44-9:~/Desktop$ mkdir dom-testing
```

1.3 Install npm using the following command:

curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.1/install.sh | bash

source ~/.bashrc # or source ~/.zshrc

nvm install --lts

nvm use --lts

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

labuser@ip-172-31-44-9:~/Desktop/dom-testing$ curl -fsSL https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.1/install.sh | bash
source ~/.bashrc  # or source ~/.zshrc
nvm install --lts
nvm use --lts
```

1.4 Run the following command and accept the default settings to initialize a new Node.js project:

npm init -y

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    SQL CONSOLE

labuser@ubuntu2204:~/Desktop/dom-testing$ npm init -y
Wrote to /home/labuser/Desktop/dom-testing/package.json:

{
  "name": "dom-testing",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

labuser@ubuntu2204:~/Desktop/dom-testing$
```

1.5 Run the following command to install Jest as a development dependency along with **jsdom**, which provides an environment for testing:
npm install --save-dev jest jsdom

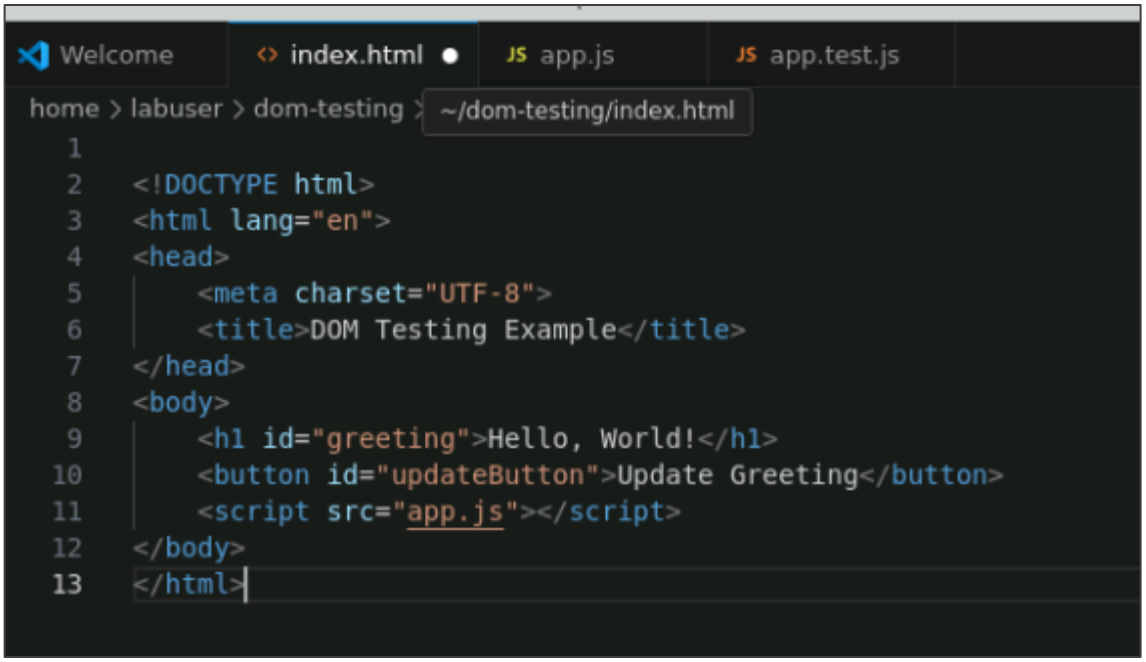
```
labuser@ip-172-31-44-9:~/Desktop/dom-testing$ npm install --save-dev jest jsdom
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do no
to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported

added 321 packages, and audited 322 packages in 15s
```

Step 2: Create an HTML file

2.1 Create a file named **index.html** in the dom-testing directory and add the following code to it:

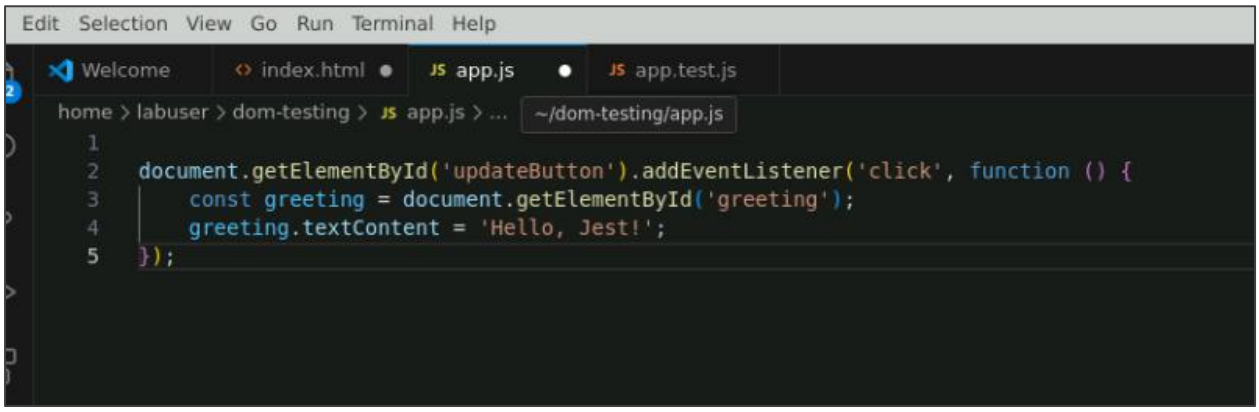
```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DOM Testing Example</title>
</head>
<body>
  <h1 id="greeting">Hello, World!</h1>
  <button id="updateButton">Update Greeting</button>
  <script src="app.js"></script>
</body>
</html>
```



Step 3: Create a JavaScript file

3.1 Create a file named **app.js** in your dom-testing directory and add the following JavaScript code to it:

```
document.getElementById('updateButton').addEventListener('click', function () {
  const greeting = document.getElementById('greeting');
  greeting.textContent = 'Hello, Jest!';
});
```



Step 4: Write a Jest test

4.1 Create a file named **app.test.js** in your dom-testing directory and add the following Jest test code:

```
const fs = require('fs');
const path = require('path');
const { JSDOM } = require('jsdom');

// Read the HTML file and set up a DOM environment
const html = fs.readFileSync(path.resolve(__dirname, '/home/labuser/Desktop/dom-
testing/index.html'), 'utf8');
const { window } = new JSDOM(html);
const { document } = window;

// Load the app.js file and attach it to the window
const appJsPath = path.resolve(__dirname, '/home/labuser/Desktop/dom-
testing/app.js');
const appJsCode = fs.readFileSync(appJsPath, 'utf8');
const script = document.createElement('script');
script.textContent = appJsCode;
document.body.appendChild(script);

// Jest test
test('Update button updates greeting', () => {
  const updateButton = document.getElementById('updateButton');
  const greeting = document.getElementById('greeting');

  // Initial state
  expect(greeting.textContent).toBe('Hello, World!');

  // Simulate a button click
  updateButton.click();

  // After click
  expect(greeting.textContent).toBe('Hello, World!');
});
```

```
Welcome   index.html ●   app.js ●   app.test.js ●
home > labuser > dom-testing > app.test.js > ...        ~/dom-testing/app.test.js
1   const fs = require('fs');
2   const path = require('path');
3   const { JSDOM } = require('jsdom');
4
5   // Read the HTML file and set up a DOM environment
6   const html = fs.readFileSync(path.resolve(__dirname, '/home/labuser/Desktop/dom-testing/index.html'), 'utf8');
7   const { window } = new JSDOM(html);
8   const { document } = window;
9
10  // Load the app.js file and attach it to the window
11  const appJsPath = path.resolve(__dirname, '/home/labuser/Desktop/dom-testing/app.js');
12  const appJsCode = fs.readFileSync(appJsPath, 'utf8');
13  const script = document.createElement('script');
14  script.textContent = appJsCode;
15  document.body.appendChild(script);
```

Step 5: Run the Jest test

5.1 In your project directory, run Jest using the following command:
npx jest

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   SQL CONSOLE

labuser@ubuntu2204:~/Desktop/dom-testing$ npx jest
 PASS  ./app.test.js
   ✓ Update button updates greeting (4 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        1.054 s, estimated 2 s
Ran all test suites.
labuser@ubuntu2204:~/Desktop/dom-testing$
```

Note: Jest will discover and execute your tests. If everything is set up correctly, you will see an output indicating that the test passed.

By completing these steps, you have successfully set up a document object model (DOM) using Jest and jsdom in a Node.js environment, created a simple interactive HTML page, and written Jest tests to verify DOM manipulations.